```
        bool k=(x&(1ll<<i));
        if(!rec[p] or !trie[p][k])return;
        p=trie[p][k];
        rec[p]--;
    }
}
inline ll search(ll x) {
    int p=1;
    ll s=0;
    for(reg int i=62;i>=0;i--) {
        bool k=(x&(1ll<<i));
        s<<=1ll;
        int &to=③;
        if(!to or !rec[to]) {
            if(rec[trie[p][k]] and trie[p][k])p=trie[p][k];
            else break;
        }
        else p=to,s|=1ll;
    }
    return s;
}
int main() {
    n=read();
    F(i,1,n)a[i]=read();
    for(reg int i=n; i>=1; i--)bk[i]=④,insert(bk[i]);
    ll cur=0;
    ans=search(cur);
    F(i,1,n) {
        cur^=a[i];
        ⑤;
        ans=max(ans,search(cur));
    }
    printf("%lld",ans);
    return 0;
}
inline ll read() { //快速读入
    reg ll x=0;
    reg char c=getchar();
    while(!isdigit(c))c=getchar();
    while(isdigit(c))x=(x<<3ll)+(x<<1ll)+(c^48),c=getchar();
    return x;
}
```

38. ①处应填（　　）。

A. `1ll<<i`　　B. `1<<i`　　C. `pow(4,i-1)`　　D. `pow(4ll,i-1)`

39. ②处应填（　　）。

A. 不填写　　B. `rec[p]++`　　C. `rec[p]--`　　D. `rec[p]^=1`

40. ③处应填（　　）。

A. `trie[p][k]`　　B. `trie[p][k^1]`

C. `++cnt`　　D. `--cnt`

41. ④处应填（　　）。

A. `bk[i]=bk[i+1]^a[i]`　　B. `bk[i]=bk[i+1]^a[i+1]`

C. `bk[i]=bk[i-1]^a[i]`　　D. `bk[i]=bk[i-1]^a[i-1]`

42. ⑤处应填（　　）。

A. `del(i)`　　B. `del(bk[i+1])`

C. `del(bk[i])`　　D. `del(bk[i-1])`